

Bilag til synopse — *Fælles aner*

[projektdeltagere]

Bilag A. Programstruktur og MVC-arkitektur

Projektet er opdelt i de tre MVC-lag, som opgaven kræver (Projekt.docx). Lagene kommunikerer i én retning: modellen beregner, controlleren orkestrerer, og views får færdige værdier og præsenterer dem.

Lag	Moduler	Rolle
Model	person.py, graph_model.py,	Data (Person, FamilyGraph) og algoritmer
View	cluster_map.py, search.py draw.py, view.py	(klusterkort, rekursiv søgning) Lagdelt Sugiyama-layout, Mermaid/Graphviz-uddrag, matplotlib-figurer
Controller	controller.py, __main__.py	FamilyController-facade + indgangspunkt, der driver pipelinen

Kørsel: `poetry run python src` (svarende til `poetry run python -m src`). Programmet skriver desuden figurer til `out/cluster_<id>.png`.

Bilag B. Kodeeksempler

B1. Datamodellen: klasser og OOP (src/person.py)

Pronouns er en *frossen* dataklasse: attributterne kan ikke ændres efter oprettelsen (immutabilitet), og PRESETS er en klassevariabel, som deles af alle instanser.

```
1 @dataclass(frozen=True)
2 class Pronouns:
3     subject: str
4     object: str
5     possessive_adjective: str
6     possessive_pronoun: str
7     reflexive: str
8
9     PRESETS: ClassVar[dict[str, Pronouns]] = {}
```

Person er den mutable modpart: en person skal kunne tildeles forældre under opbygningen af familiedata. Hvert objekt gemmer højst to forældreferencer (mom/dad), hvilket er invariant **I1** fra designdokumentet: $\deg^-(v) \leq 2$. Specialmetoden `__str__` gør objektet læsbart ved udskrift.

```

1 class Person:
2     def __init__(
3         self,
4         name: str = "John Doe",
5         mom: Person | None = None,
6         dad: Person | None = None,
7         pronouns: Pronouns | None = None,
8     ):
9         self.mom = mom
10        self.dad = dad
11        self.name = name
12        self.pronouns = pronouns or Pronouns.from_preset("they")
13
14    def __str__(self) -> str:
15        known: list[str] = []
16        if self.mom:
17            known.append(f"mother is {self.mom.name}")
18        if self.dad:
19            known.append(f"father is {self.dad.name}")
20
21        if not known:
22            return f"{self.name} has no known parents."
23        if len(known) == 1:
24            # exactly one parent known: name the *other* one as
25            # the unknown slot
26            known.append("no known father" if self.mom else "no
27                known mother")
28
29        return f"{self.name}'s " + " and ".join(known) + "."

```

B2. Grafrepræsentationen (src/graph_model.py)

FamilyGraph er en frossen dataklasse med tætte heltals-id'er (O4) og et omvendt forældre-til-barn-index, som bygges *en gang* (O1). Nabolaget i den udretningsløse skygge $\bar{E}$ er forældre $\cup$ børn — klustertilhørighed er ligeglad med orientering.

```

1     def neighbours(self, p: Person) -> tuple[Person, ...]:
2         # Undirected neighbourhood in E-bar (parents u children),
3         # self-loops dropped.
4         seen: set[Person] = {p}
5         out: list[Person] = []
6         for q in (p.mom, p.dad, *self.child_index.get(p, ())):
7             if q is not None and q not in seen:
8                 seen.add(q)
9                 out.append(q)
10        return tuple(out)

```

B3. Rekursiv søgning som mindste fast punkt (src/search.py)

Klusteret omkring en start-person v er det *mindste faste punkt* for operatoren

$$\Phi_v(S) = \{v\} \cup \bigcup_{\{u,w\} \in \bar{E}, u \in S} \{w\}.$$

(Knaster–Tarski). `visited`-vagten er netop fast-punkt-stabiliseringen: hvert knudepunkt udvides præcis én gang, så $T = \Theta(n + m) = \Theta(n)$.

```
1 def explore(  
2     node: Any,  
3     visited: set[Any],  
4     members: list[Any],  
5     neighbours: NeighboursFn | None = None,  
6 ) -> None:  
7     if node is None or node in visited:  
8         return  
9     visited.add(node)  
10    members.append(node)  
11    for w in _neighbours(node, neighbours):  
12        explore(w, visited, members, neighbours)
```

Rekursionen er den læsbare reference. Produktionsvejen (O5) er den iterative variant, fordi en ren kæde har dybde $\Theta(n)$, mens CPython kun har $R = 1000$ rammer:

```
1 def explore_iterative(  
2     node: Any,  
3     members: set[Any] | None = None,  
4     neighbours: NeighboursFn | None = None,  
5 ) -> set[Any]:  
6     seen = members if members is not None else set()  
7     if node is None:  
8         return seen  
9     stack = [node]  
10    while stack:  
11        current = stack.pop()  
12        if current is None or current in seen:  
13            continue  
14        seen.add(current)  
15        stack.extend(w for w in _neighbours(current, neighbours)  
16                     if w not in seen)  
17    return seen
```

B4. Union-Find og identitet (3) (src/cluster_map.py)

Union-Find med union-efter-rang og path-halving forbinder kanter i nær-konstant amortiseret tid. Antallet af klustre følger den afkassede identitet (3) (Theorem 3.3):

$$k = n - s, \quad s = \text{antal vellykkede union-kald.}$$

```

1 class UnionFind:
2     __slots__ = ("parent", "rank", "successful_unions")
3
4     def __init__(self, n: int):
5         self.parent = list(range(n))
6         self.rank = [0] * n
7         self.successful_unions = 0
8
9     def find(self, x: int) -> int:
10        while self.parent[x] != x:
11            self.parent[x] = self.parent[self.parent[x]] # path
12                halving (03)
13            x = self.parent[x]
14        return x
15
16    def union(self, a: int, b: int) -> bool:
17        ra, rb = self.find(a), self.find(b)
18        if ra == rb:
19            return False
20        if self.rank[ra] < self.rank[rb]: # union by rank: serve
21            the shallower root
22            ra, rb = rb, ra
23        self.parent[rb] = ra
24        if self.rank[ra] == self.rank[rb]:
25            self.rank[ra] += 1
26        self.successful_unions += 1
27        return True

```

B5. Memoisering af forfædre-udregningen (src/search.py)

Uden cache er rekursionen $\text{Anc}(v) = \{v\} \cup \text{Anc}(\text{mom}(v)) \cup \text{Anc}(\text{dad}(v))$ eksponentiel ved pedigree-collapse (I3): $\Theta(\varphi^d)$ for Fibonacci-stamtavler. Med memoisering beregnes hvert knudepunkt præcis én gang, så $T = \Theta(n + m)$.

```

1 def ancestors(p: Any) -> frozenset[Any]:
2     cache: dict[Any, frozenset[Any]] = {}
3
4     def closure(node: Any) -> frozenset[Any]:
5         if node is None:
6             return frozenset()
7         hit = cache.get(node)
8         if hit is not None:
9             return hit
10        out = {node} | set(closure(node.mom)) | set(closure(node.
11            dad))
12        cache[node] = frozenset(out)
13        return cache[node]
14
15    return closure(p)

```

Den iterative variant `ancestors_iterative` beregner samme lukning med eksplicit stak (O5) og rejser `CycleError` ved forældrecyklusser i stedet for at køre fast.

B6. Tegning af klusterkortet: Sugiyama-layout (`src/draw.py`)

Første gennemløb tildeler generationer som længste-vej-rang (Kahn's algoritme); et barn ligger præcis én generationsskridt under begge forældre. Et cyklus-tjek rejses *før* layoutet, hvis data ikke opfylder invariant **I2**.

```
1 ranks = {v: 0 for v in members}
2 indegree = {v: len(parents[v]) for v in members}
3 ready = deque(
4     sorted((v for v in members if indegree[v] == 0), key=
5         lambda p: p.name)
6 )
7 processed = 0
8 while ready:
9     parent = ready.popleft()
10    processed += 1
11    for child in children[parent]:
12        ranks[child] = max(
13            ranks[child], ranks[parent] + delta
14        ) # longest path (§6.1)
15        indegree[child] -= 1
16        if indegree[child] == 0:
17            ready.append(child)
18 if processed != len(members):
19     raise CycleError("cycle in parental record -- layered
20         layout needs a DAG (I2)")
```

Andet gennemløb minimerer kanter der krydser hinanden med barycenter-heuristikken $b(u) = \frac{1}{\deg(u)} \sum_{w \in E(u)} \text{pos}(w)$ — problemet er NP-hardt i almindelighed, så en heuristik er den accepterede løsning:

```
1 def bary(node: Any) -> tuple[float, int]:
2     nbs = neighbours[node]
3     if not nbs:
4         return float(pos[node]), pos[node] # isolated:
5         keep position
6     b = sum(pos[w] for w in nbs if w in pos) / len(nbs)
7     return b, pos[node]
8
9 nodes.sort(key=bary) # stable: ties broken by current
10 position
```

B7. Visualisering med `matplotlib.pyplot` (`src/view.py`)

View-laget oversætter layout-koordinaterne til en figur: én boks pr. person ved $(x, y) = (\text{position i laget, rang})$, og en pil pr. forældrereferens fra forælder til barn — samme orientering som Mermaid- og Graphviz-uddragene.

```

1     for child, parent in cluster.edges: # (child, parent) per §
2         1.1
3         ax.annotate(
4             "",
5             xy=(layout.x[child] * X_GAP, layout.ranks[child]),
6             xytext=(layout.x[parent] * X_GAP, layout.ranks[parent
7                 ]),
8             arrowprops={"arrowstyle": "->", "color": EDGE_COLOR,
9                 "lw": 1.2,
10                "shrinkA": 10, "shrinkB": 10},
11         )
12     for person in members:
13         ax.text(
14             layout.x[person] * X_GAP,
15             layout.ranks[person],
16             label(person),
17             ha="center",
18             va="center",
19             fontsize=9,
20             bbox=HIGHLIGHT_STYLE if person in hot else NODE_STYLE
21         )

```

B8. Fælles aner og slægtskab (projekt 7.10)

Opgavens kerne i afsnit 7.10 er at besvare spørgsmålet “*Har to personer mindst en fælles ane?*”. Svaret er skæringspunktet af de to forfædre-mængder $\text{Anc}(a) \cap \text{Anc}(b)$: en tom mængde betyder ikke-slægtskab. Fordi $\text{Anc}(v)$ per §2.5 også indeholder personen selv, tæller en forælder og et barn som beslægtede — forælderen er en fælles “ane” for begge, hvilket er den semantik, spørgsmålet forudsætter.

```

1 def find_common_ancestor(a: Any, b: Any) -> list[Any]:
2     """Every common ancestor of a and b as a list (project
3         requirement 7.10)."""
4     if a is None or b is None:
5         return []
6     shared = ancestors_iterative(a) & ancestors_iterative(b)
7     return sorted(shared, key=lambda p: getattr(p, "name", str(p)))
8
9 def is_related(a: Any, b: Any) -> bool:
10    """True iff a and b share at least one common ancestor (7.10)
11        ."""
12    if a is None or b is None:
13        return False
14    return not ancestors_iterative(a).isdisjoint(
15        ancestors_iterative(b))

```

Begge funktioner er memoiserede (to $\Theta(n + m)$ -opslag plus mængde- operationen) og eksponeres gennem controlleren som `find_common_ancestor()` og `is_related()`.

Bilag C. Visualisering af træet

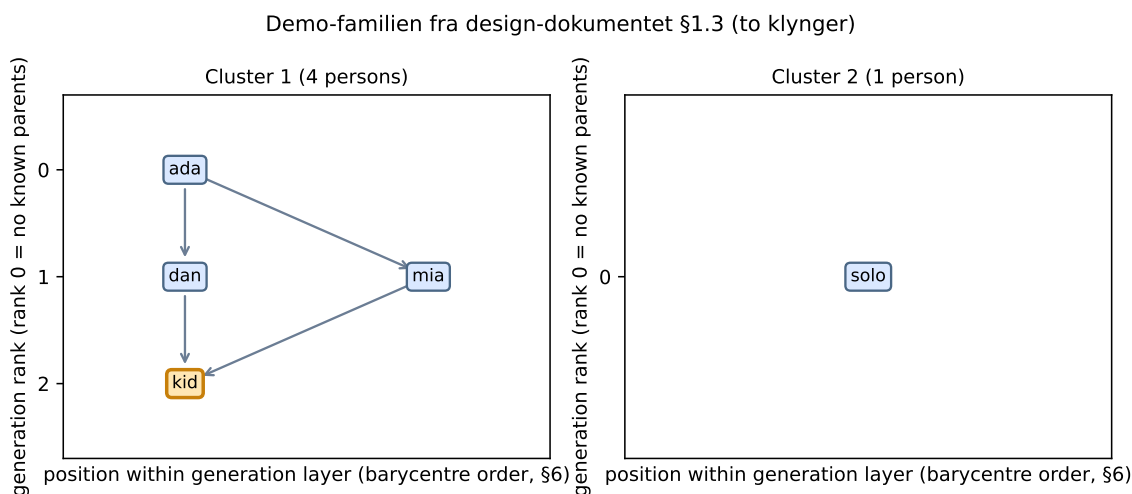


Figure 1: Demo-familien fra design-dokumentet §1.3 tegnet af programmets eget view-modul. To klynger: sammenbrudsfamilien til venstre (ada er forælder til både mia og dan, hvilket er pedigree-collapse, invariant **I3**) og den isolerede person solo til højre. Søgeudgangspunktet kid er fremhævet.

Bilag D. Udviklingsprocessen (Git)

Koden er udviklet på `dev`-branchen i repoet

https://github.com/ExperiencesXP/proj_heritage_tree

Commit-oversigten pr. 2026-10-07:

Hash	Dato	Besked
c163937	2026-10-07 19:49	docs: fix clone URL and document running, tests and MVC layers
c35db37	2026-10-07 19:49	feat: add matplotlib visualization and MVC controller
893432f	2026-10-07 19:49	fix: use visited-guarded goal search over the undirected shadow
094749d	2026-10-07 19:44	fix: repair <code>__str__</code> parent-slot text and pairs-map arrow orientation
f7fc0e7	2026-10-05 13:45	works on my machine bro
8dd1ba4	2026-10-01 11:56	Wire the finalized entry point: <code>poetry run python src</code>
47d965a	2026-10-01 11:27	Implement cluster map and recursive search per design doc
20e2447	2026-10-01 10:02	do this and that
f724764	2026-09-29 21:45	becoming woke
a8c0032	2026-09-28 15:16	Initial commit

SKÆRMDUMP 1 (indsat her): Git Graph i VS Code (udvidelsen *Git Graph*) med grenen `dev` og alle commits.

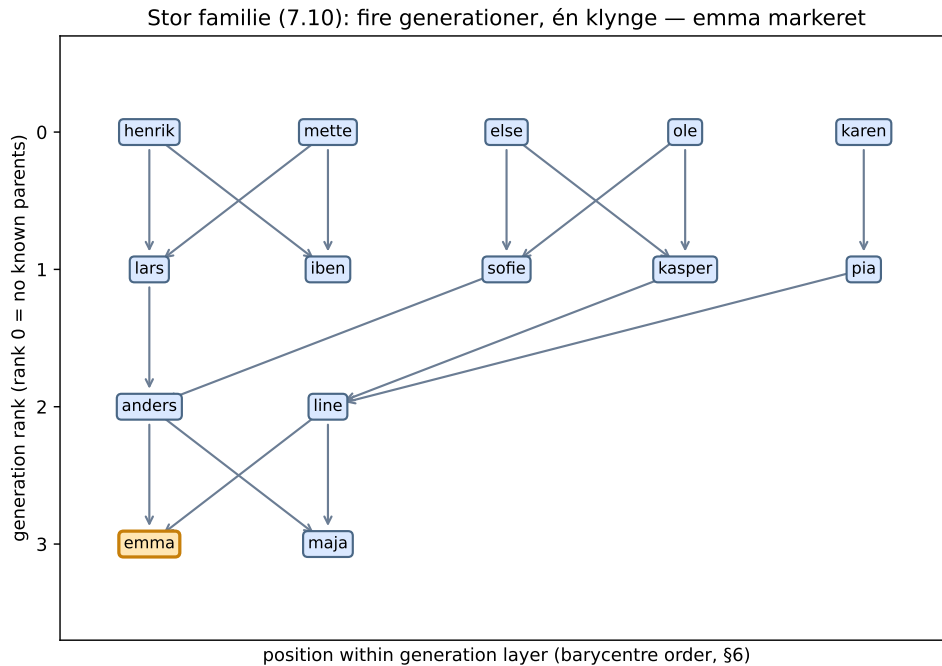


Figure 2: Den “store familie” fra opgave 7.10: 14 personer, fire generationer, én klynge. Anders’ mor sofie og lines far kasper er søskende, så deres børn emma og maja har et sammenbrudt stamtræ på ole/else — programmets eget slægtskabsspørgsmål finder præcis de to som fælles aner for anders og line. emma er fremhævet.

SKÆRMDUMP 2 (indsat her): commit-oversigt med datoer —
https://github.com/ExperiencesXP/proj_heritage_tree/commits/dev.

Bilag E. Verifikation: kørsel og tests

`poetry run pytest` giver 21 tests, som dækker verifikationsplanens §8: klusterkort sammenlignes med en uafhængig BFS-reference, identitet (3) $k = n - s$ holdes på tilfældige populationer, dybe kæder klarer de iterative traverseringer, cyklusser afvises, og skaleringen er nær-lineær.

Udskrift fra `poetry run python src` (uddrag):

```

1 Heritage-tree cluster map (docs/cluster_map_and_recursive_search.md)
2 population n = 5, parent edges m = 4 (I1: m <= 2n)
3 clusters k = 2 (identity 3: k = n - s with s = 3)
4   C2: 1 persons - solo
5   C1: 4 persons - ada, dan, kid, mia
6 cluster of kid (explore_iterative over G-bar): ada, dan, kid, mia
7 ancestors of kid: 4 - ada, dan, kid, mia (pedigree collapse reduces
8 the binary-lattice bound)
9 descendants of ada: 3 - dan, kid, mia
10 early-exit search found: ada (07)
11
12 layered layout (ranks and crossings counts):
13   C1: ranks [0, 1, 2], crossings = 1
14   C2: ranks [0], crossings = 0

```

Bemærk identiteten i praksis: $n = 5$, $m = 4$ forældreferencer, $s = 3$ vellykkede sammenlægninger, $k = 2$ klynger — og $k = n - s = 5 - 3 = 2$.

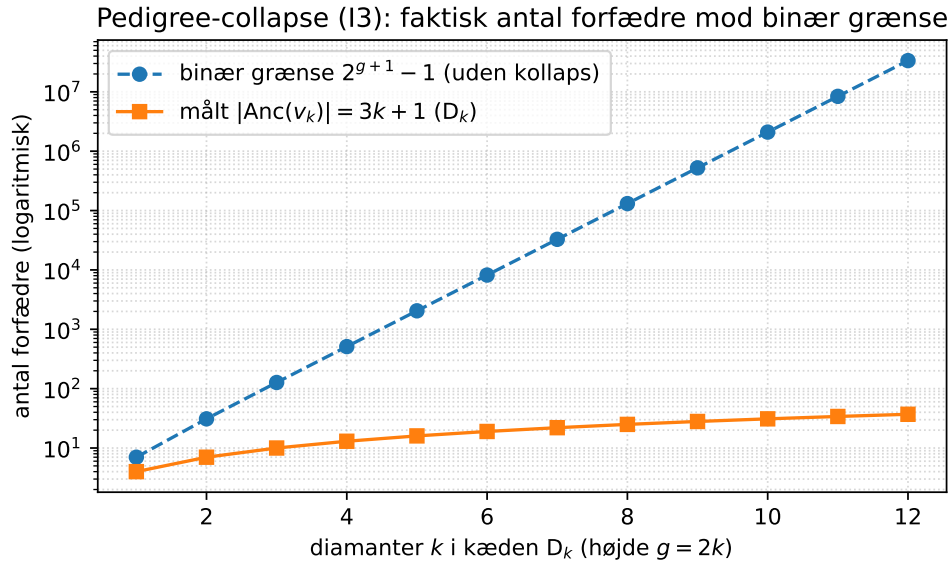


Figure 3: Pedigree-collapse i tal: for diamanstkæden D_k er det faktiske antal forfædre $|\text{Anc}(v_k)| = 3k + 1$, mens den naive binære grænse $2^{g+1} - 1$ vokser eksponentielt med højden $g = 2k$. Målt med programmets `ancestors_iterative`.

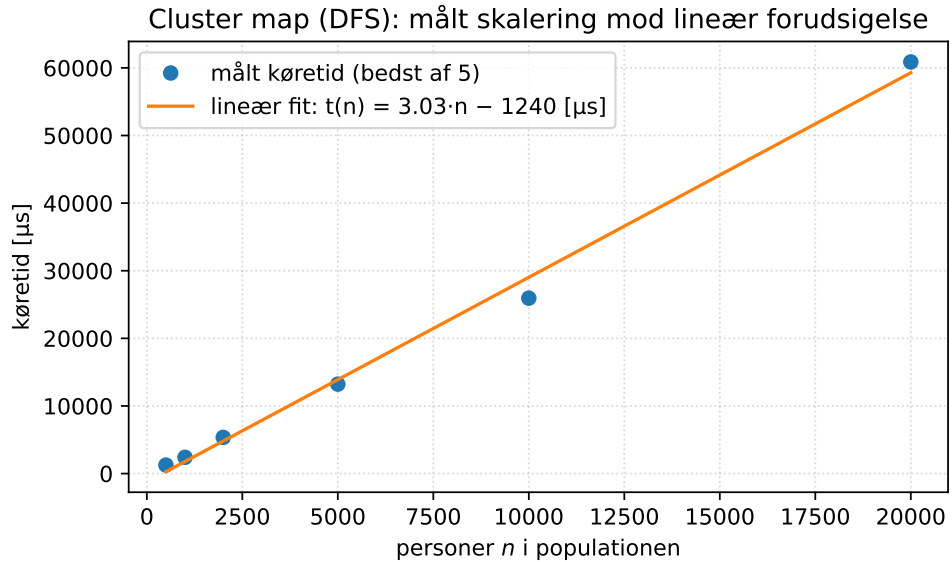


Figure 4: Målt køretid for klusterkort-konstruktionen (DFS) vha. `time.perf_counter`, bedst af 5 gentagelser pr. punkt. Den lineære fit $t(n) \approx c \cdot n$ bekræfter kompleksitetsanalysen $T = \Theta(n + m) = \Theta(n)$ fra §4.1 (invariant **I1** giver $m \leq 2n$).